

Brutal Judge Review of APEX for Best Use of Technology

The judge lens that matters here

If I were scoring this as a hard-nosed BeMyApp / IBM SkillsBuild judge, I would not grade it on how many IBM names appear on the landing page. I would grade it on the published rubric: **Technical Execution** means effective use of IBM and open source technologies in a **functional, well-structured** solution; **Innovation** means originality; **Challenge Fit** means relevance to racing; and **Implementation & Feasibility** means practicality and real-world usability. The contest rules also require a public GitHub repo, a functioning prototype or proof of concept, a clear README, and a solution video. “Best Use of Technology” is a sponsor track prize, but the core rules still anchor judging in execution and feasibility, not stack size.

1

That matters because APEX is not a shallow idea. The adaptive-driver angle is real, the racing fit is strong, and the team has clearly thought harder than most hackathon entries about constraint handling, regulatory inputs, and safety contracts. The public materials also do something rare and commendable: they explicitly label what is wired, what is “integration,” what is a facade, and what is still pending. That honesty helps trust. It does **not**, however, rescue the score if the deepest IBM claims are still more architectural than operational by judging day.

2

What is genuinely strong before the knives come out

The strongest thing about APEX is that it is chasing a meaningful gap, not a toy problem. FIA-backed materials make clear that disabled drivers and adapted vehicles are a real motorsport audience, with formal guidance around vehicle adaptations and hand-control systems, not a fictional niche invented for a hackathon. APEX's core thesis — that mainstream coaching workflows assume able-bodied control envelopes and can misread adapted driving inputs — is directionally serious and challenge-relevant.

3

The second real strength is conceptual architecture. The public architecture spec does not just say “AI analyzes telemetry.” It defines a frozen time-series forecaster, then wraps it in a projection-and-audit layer, then serializes violations for a downstream safety gate, then feeds that into a narrator. That is a more defensible system design than the typical “LLM summarizes racing data” submission, and it is exactly why APEX is worth criticizing seriously rather than dismissing outright.

4

Ranked brutal criticisms

Mostly a concept stack, not a shipped stack

This is the biggest score killer.

APEX's own public surfaces say that only **two** tools are fully wired at HEAD: Granite Instruct 4.1 8B and Granite 4.0 Nano 350M. The rest are described as "integration," "accelerator," "demo-facade," or "frontend-integration." The judges page also says the LangGraph + MCP + ContextForge runtime is currently shown through a **canned-fallback engine**, the in-browser TTM surface is also **canned-fallback**, the TSPulse section uses **mock fixtures today**, and the more advanced Stage A / Stage B physics traces are still canned while real runtime wiring is pending. On the same judges page, the submission video, 30-second highlight, and pitch deck are still marked **Pending**. The official rules, meanwhile, require a functioning prototype and a 3-minute solution presentation video. That combination is deadly for a "Best Use of Technology" pitch because a tired judge will read it as: "great architecture, but the IBM stack is still mostly in proposal mode."

5

A brutal judge would say this out loud: **"You are asking me to reward planned integration, not executed integration."** The honesty about status helps, but it also gives the judge exact language to score you down on technical execution and implementation. If the orchestration runtime is canned, the time-series browser surface is canned, the anomaly panels are mock-backed, and core media assets are pending, then "12-tool stack" reads more like a roadmap than a finished technology showcase. 6

The fix before **2026-05-31** is not "wire everything." That is too ambitious and the wrong instinct. The fix is to **brutally shrink the headline** to the tools that are truly end-to-end and load-bearing in user-visible flow. If LangGraph, FlowState, TSPulse, and the in-browser TTM path are not truly live, take them out of the hero narrative and move them to an "in progress / beyond submission" appendix. Then, for the sponsor-track pitch, make the public demo undeniably real with five things only: COA parse, TTM forecast, physics projection, Guardian audit, Instruct narration. Replace every mock or canned panel on `/judges` with either a live result or a clearly labeled artifact replay produced by the actual backend. If the video and deck are not live by submission, that is not a sponsor-track problem, that is a submission-readiness problem. 7

The COA document story collapses to one bit and risks looking shallow

This is the second-biggest problem because it hits the project's most distinctive claim.

APEX repeatedly positions itself as the first workflow that reads the FIA Certificate of Adaptations as a **binding regulatory input at the tensor level**. But the architecture spec makes clear what that tensor-level regulatory input actually is: the ninth synthetic channel is a **binary COA simultaneity bit**, `coa_simul_permitted`, derived from the COA parse. The same document then says the novelty is that the parsed COA JSON is **reduced to a binary flag** occupying the ninth channel of the TTM input tensor. That is much less impressive than the headline sounds. A brutal judge will phrase it like this: **"You built a sophisticated PDF-to-one-bit pipeline and are calling it deep document integration."** 8

The problem gets worse because APEX counts **Granite-Docling** and the **Docling library** as separate tools in the 12-tool story. Yet the architecture spec says the Docling library's role is to post-process Granite-Docling output into a structured schema, and the main tensor-level effect that the race-engineering pipeline publicly emphasizes is still the binary overlap flag. Docling itself is a broad document-processing framework with rich PDF understanding, OCR, reading order, tables, formulas, and structured export capabilities; Granite-Docling is built for document conversion and full compatibility with DoclingDocuments. Against that backdrop, using both technologies to derive one binary constraint makes the pair look underexploited and, frankly, a little like stack padding. 9

The fix before **2026-05-31** is to make the COA genuinely multi-parameter and visibly load-bearing. Right now the system should surface at least three to five **live** COA-derived constraints or features, not one. Examples that would read as real depth: brake travel bounds, steering adaptation limits, seating/headrest constraints that alter recommended setup targets, homologation-linked equipment constraints that affect advisory language, and explicit citation/provenance from PDF section → structured JSON field → tensor or rule input → coaching output. If you cannot ship multi-parameter use in time, then narrow the headline. Say what is true: **“APEX uses Granite-Docling to derive a COA-controlled brake/throttle simultaneity gate.”** That is still interesting. Overselling it as deep tensorized regulatory conditioning across the full document is exactly the kind of phrasing a brutal judge punishes. ⁸

The TSFM story does not yet fully line up with the inspectable implementation

This is where an IBM-savvy judge will get suspicious.

Public IBM and Hugging Face documentation say Granite TTM r2 / r2.1 supports forecasting at **minutely to hourly** resolutions, with r2.1 adding **daily and weekly** support; the APEX architecture spec explicitly acknowledges that TTM does **not** support sub-second resolution and says the team aggregates raw **50 Hz** telemetry to **1 Hz mini-sector** tensors to fit the model's support envelope. That is a clever adaptation. It is also a massive lossy transformation of racing telemetry. A brutal judge will ask the obvious question: **“How much racing signal did you throw away just to force-fit the IBM model you wanted to use?”** Without hard experiments, the answer is unknown. ¹⁰

The second crack is that the public implementation surface does not cleanly support the marketed model story. The README and judges page talk about a frozen-backbone TTM with a **Track 1 channel-mix decoder fine-tune**, but the publicly inspectable `forecast.py` path loads `ibm-granite/granite-timeseries-ttm-r2` directly from pretrained weights and describes itself as a frozen zero-shot forecaster. That does not prove there is no fine-tune elsewhere in the repo, but it absolutely means the **main inference path a judge can verify quickly** looks like the base model, not a visible fine-tuned checkpoint or a clearly wired channel-mixing deployment path. In a 30-second review, that mismatch hurts. ¹¹

Then there is FlowState. IBM's own documentation presents FlowState as time-scale adjustable and suitable for forecasting across sampling rates, but an IBM Granite discussion on the official model page says that while the model can be used for multivariate forecasting, the **currently available code** only showcased **univariate** usage and multivariate enablement was still planned. APEX, however, publicly claims FlowState 9.1M as **Track 2** integration at **50 Hz**. That is exactly the sort of thing a judge flags as “promising, but not yet defensible on shipped evidence.” The same vibe attaches to TSPulse: IBM positions it strongly for anomaly detection, but APEX's own judges surface says its TSPulse panels are mock-backed “today.” ¹²

The fix before **2026-05-31** is to pick one of two lanes and stop straddling both. Lane one: **prove the current TTM lane**. Publish the actual checkpoint or adapter used in inference, show a tiny but real holdout evaluation, and demonstrate that the 1 Hz mini-sector aggregation preserves enough signal to beat simpler baselines or at least produce better downstream advice than naive forecasting. Lane two: **de-escalate the claim**. Admit that submission-day APEX uses a frozen base TTM with a projection layer, while FlowState and TSPulse are forward paths not yet sponsor-track-critical. That narrower claim is less flashy, but much more scoreable. ¹³

The Guardian and D-050 story is overmarketed relative to what is actually proven

This is the criticism Stephen most needs to hear, because it targets the project's favorite internal "killshot."

Granite Guardian 4.1 8B is an actual IBM safety model designed to judge whether prompts and outputs of LLM systems satisfy specified criteria, including hallucination-related criteria for tool calls and RAG workflows. But APEX's own shipped `audit.py` says the **actual Granite Guardian model integration lands later**, and that the current module is a **deterministic rule-engine floor** that the Guardian model will eventually wrap. In other words, the public code path that a judge can verify today is not really "Granite Guardian is doing the safety work"; it is "a deterministic rules layer exists now, and Granite Guardian is part of the planned wrapper story." For Best Use of Technology, that is a meaningful downgrade. ¹⁴

The same problem applies to the D-050 byte-equality headline. The tests do show something real: the V2 `cvxpylayers` projector test asserts byte-equality with the V1 NumPy path modulo the engine line. But that same test file says the more ambitious **8-tier Pacejka + 3-iteration unroll** work is deferred, and the judges page says Stage A and Stage B still ship as **canned-fallback** traces at HEAD. So the current proof is narrower than the marketing. It is a useful regression invariant on a constrained path, not yet a decisive proof that the full engine-swapped physics stack preserves user-relevant outputs under real runtime conditions. A brutal judge will say: **"Byte-identical logs are nice plumbing. Show me byte-identical trustworthiness or coaching outcomes."** ¹⁵

The deeper issue is strategic, not just technical. APEX currently treats D-050 as a load-bearing *positioning claim*. Judges do not reward internal elegance unless it cashes out into external reliability. Right now the byte-equality result proves serializer compatibility far more than it proves better racing advice, safer decisions, or better IBM model usage. That is meaningful engineering, but it is not sponsor-track-winning storytelling unless paired with **recommendation-level invariance, latency deltas, and decision-quality deltas**. ¹⁶

The fix before **2026-05-31** is to reframe and extend. Reframe D-050 as a **regression guarantee**, not the project's headline. Then extend the evidence: show that V1 and V2 preserve the same **verdict, blocked recommendation set, and driver-facing recommendation** on the same fixture; wire the real Granite Guardian pass so it checks not only structured violation logs but also whether the narrator invents citations or softens unsafe language; and publish two or three concrete failure cases that only the combined Guardian-plus-rules path catches. That would turn "nice internal invariant" into "defensible IBM safety integration." ¹⁷

There is not enough judge-facing evidence that every extra IBM tool earns its place

This is where "Best Use of Technology" can quietly slip away even if the idea is admired.

The judges page itself says the NeurIPS workshop draft is live but that **54 Experiments** will be filled "at camera-ready," and the paper draft repeats that the experiments section is still outline-plus-tables-to-be-reported. The same judges page says some panels use **mock fixtures today**. That means the public evidence package is still stronger on architecture prose than on comparative results. For a normal hackathon, that might be acceptable. For a sponsor-track prize explicitly about technology use, it is not enough. Without ablations and measured deltas, tools like TSPulse, Embedding R2, LangGraph, Granite

MCP Gateway, ContextForge, WebGPU TTM, and IBM Bob can read like a badge collage rather than a necessity stack. ¹⁸

IBM Bob is the cleanest example. IBM's Ferrari page does say Ferrari uses IBM Bob as an AI coding agent to improve coding efficiency and accuracy during app development. But APEX also labels Bob as an **accelerator** and "off-loop, build-time." That is fine as a development aid. It is much weaker as a sponsor-track differentiator unless the team shows that Bob materially improved reproducibility, test quality, defect rate, or delivery speed in a way the judge can see. Otherwise Bob becomes another item that inflates the stack count without moving the product score. ¹⁹

The fix before **2026-05-31** is the simplest and highest ROI of all: publish one judge-facing ablation page and one judge-facing evidence table. The ablation page should show, on the same fixture set: no COA gate vs COA gate, no projection vs projection, rules-only vs rules+Guardian, no RAG vs RAG, no anomaly detector vs anomaly detector. The evidence table should show four metrics only: latency, safety violation capture, recommendation block/allow correctness, and report usefulness. If a tool cannot show a measurable delta by deadline, demote it from the sponsor-track narrative. That single editorial decision would immediately make APEX look more like "best use" and less like "maximal use." ²⁰

Why this hurts Best Use of Technology more than the team seems to think

Best Use of Technology is not usually won by the entry with the longest model list. It is won by the entry where a judge can see, very quickly, that the chosen technologies are doing work that simpler alternatives would not have done, and that the system is already functional, coherent, and practical. The contest's own language points there: effective use, functional structure, implementation, feasibility. APEX absolutely has the seeds of that. But in its current public state, too much of the sponsor-track argument still depends on **future wiring, mock or canned surfaces, broad claims narrowed in footnotes, and internal invariants sold as external proof**. That combination is exactly how a technically ambitious project gets admired and still loses the technology prize. ²¹

The team's public confidence signal also actively creates risk. The site shows internal calibration numbers like "Best Use of Technology 96%," while the same public surface says only two tools are wired and multiple major pieces are pending or mock-backed. A brutal judge will see that as self-hype outrunning shipped evidence. In sponsor-track judging, that almost always produces a harsher reaction than modesty would. ²²

Bottom line

If I were judging **today**, I would **not** put APEX first for Best Use of Technology. I would probably score it as **strong on originality and challenge fit**, but materially lower on **technical execution and implementation/feasibility** than the team appears to expect, because the current public evidence shows a conceptually excellent system whose most ambitious IBM integrations are still only partially operational. ²³

The good news is that this is still salvageable before **2026-05-31**. The path to winning is not "add more." It is the opposite: **trim the claim set, wire the core path, publish ablations, turn the COA from one bit into**

several active constraints if possible, and stop selling D-050 as if it were user value. If APEX shows five truly live IBM components doing unmistakably necessary work in one end-to-end flow, it becomes a sponsor-track contender. If it ships as a 12-tool architecture opera with eight tools still half-real, it will lose to something simpler that is more obviously done. ²⁴

Open questions and limits

This review is based on the **public** repo, paper draft, challenge rules, official IBM / Hugging Face model documentation, and the live APEX surfaces visible on **May 25, 2026**. Some APEX assets are explicitly marked pending and may change before the deadline, including the video, deck, and parts of the experimental package. I therefore treated this as a review of the project's **current public judge-facing state**, not a claim about what the team might still ship in the final six days. ²⁵

¹ ²⁰ ²¹ ²³ https://docs.google.com/document/d/e/2PACX-1vRgk2rv1IU6Y36Ef2A1C6udouFrmeRHR6TkLQgcya3W9C_5ysV8f3UvrXCH1Ldyyw/pub
https://docs.google.com/document/d/e/2PACX-1vRgk2rv1IU6Y36Ef2A1C6udouFrmeRHR6TkLQgcya3W9C_5ysV8f3UvrXCH1Ldyyw/pub

² ⁵ ²² APEX | AI race engineer for adaptive racers
<https://apex-one-black.vercel.app/>

³ <https://www.fiafoundation.org/news/fia-launches-vehicle-adaption-guidelines-for-disabled-motorsport-drivers-supported-by-fia-foundation>
<https://www.fiafoundation.org/news/fia-launches-vehicle-adaption-guidelines-for-disabled-motorsport-drivers-supported-by-fia-foundation>

⁴ ¹¹ GitHub - StephenSook/apex: AI race engineer for adaptive racers, built on IBM Granite. IBM SkillsBuild May 2026. · GitHub
<https://github.com/StephenSook/apex>

⁶ ⁷ ¹⁶ ¹⁸ ²⁴ ²⁵ Judges' Tour · APEX | APEX
<https://apex-one-black.vercel.app/judges>

⁸ ⁹ ¹³ <https://github.com/StephenSook/apex/blob/main/docs/architecture-spec.md>
<https://github.com/StephenSook/apex/blob/main/docs/architecture-spec.md>

¹⁰ <https://huggingface.co/ibm-granite/granite-timeseries-ttm-r2>
<https://huggingface.co/ibm-granite/granite-timeseries-ttm-r2>

¹² <https://huggingface.co/ibm-granite/granite-timeseries-flowstate-r1>
<https://huggingface.co/ibm-granite/granite-timeseries-flowstate-r1>

¹⁴ <https://huggingface.co/ibm-granite/granite-guardian-4.1-8b>
<https://huggingface.co/ibm-granite/granite-guardian-4.1-8b>

¹⁵ https://github.com/StephenSook/apex/raw/refs/heads/main/app/backend/tests/test_physics_v2.py
https://github.com/StephenSook/apex/raw/refs/heads/main/app/backend/tests/test_physics_v2.py

¹⁷ [raw.githubusercontent.com](https://raw.githubusercontent.com/StephenSook/apex/main/app/backend/apex/guardian/audit.py)
<https://github.com/StephenSook/apex/raw/refs/heads/main/app/backend/apex/guardian/audit.py>

19 IBM and Scuderia Ferrari

<https://www.ibm.com/sports/ferrari>